NATIONAL AND KAPODISTRIAN UNIVERSITY OF ATHENS
DEPARTMENT OF INFORMATICS AND TELECOMMUNICATIONS

Deep Learning for NLP

Student name: *Orfanos Konstantinos*
ID: sdi2300150

Course: *Artificial Intelligence II (YS19 M226 M262 M325 M405)*
Semester: *Spring Semester 2025-2026*

Contents

1 Abstract **2**

2 Data processing and analysis **3**
2.1 Pre-processing . . . 3
2.2 Analysis . . . 3
2.3 Data partitioning for train, test and validation . . . 4
2.4 Vectorization . . . 4

3 Algorithms and Experiments **5**
3.1 Experiments . . . 5
3.1.1 Table of Trials . . . 13
3.2 Hyper-parameter tuning . . . 13
3.3 Optimization techniques . . . 14
3.4 Evaluation . . . 14
3.4.1 Confusion Matrix . . . 14
3.4.2 ROC Curve . . . 15
3.4.3 Learning Curve . . . 15

4 Results and Overall Analysis **16**
4.1 Results Analysis . . . 16
4.1.1 Best trial . . . 16

5 Bibliography **17**

1. Abstract

This report describes the implementation and evaluation of text classifiers for response clarity classification in political interviews, as part of SemEval 2026 Task 6 (CLARITY) [2]. The dataset consists of question-answer pairs from U.S. presidential interviews, where the goal is to determine whether a politician's answer constitutes a clear reply, an ambivalent response, or a clear non-reply. Two approaches are compared: TF-IDF with Logistic Regression and Word2Vec with Logistic Regression. Through systematic experimentation across eleven runs, TF-IDF consistently outperformed Word2Vec, though both models exhibited significant overfitting on this small dataset — an issue that proved resistant to resolution within the constraints of these model families. The most impactful findings were that balanced class weighting and bigram features each contributed meaningfully to the final configuration.

2. Data processing and analysis

2.1. Pre-processing

All input text underwent a standardized preprocessing pipeline prior to feature extraction. The `question` and `interview_answer` fields were concatenated using a `[SEP]` separator token to form a single input string, clearly marking the boundary between the two fields. All text was then lowercased to reduce vocabulary size and ensure consistent treatment of words regardless of capitalization. Punctuation was removed as it carries no semantic value for the models used in this study. Stopword removal was deliberately omitted, as the short length of political questions means that common words may still carry structural and contextual importance — a design choice later validated empirically in Run 5, where stopword removal led to a drop in F1 for both models.

2.2. Analysis

The dataset is significantly imbalanced across the three classes. *Ambivalent* is the dominant class with 2040 samples (59.2%), followed by *Clear Reply* with 1052 samples (30.5%), and *Clear Non-Reply* being the minority class with only 356 samples (10.3%). This severe imbalance was the primary motivation for using `class_weight="balanced"` in all models.

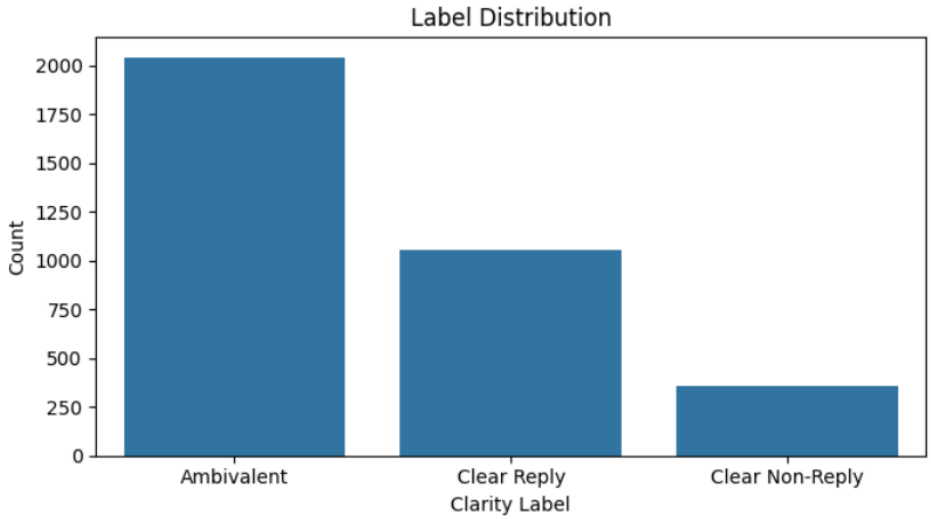

Figure 1: Label Distribution

Text length analysis reveals a stark difference between questions and answers. Questions are relatively short, averaging 14.5 words (median 12), with most falling between 8 and 19 words. Answers are substantially longer, averaging 293 words (median 207), with a wide spread up to 2117 words. This asymmetry suggests that the answer field carries significantly more information than the question, though the short question length also informed the decision to preserve stopwords, as their removal would strip too much context from already brief inputs (later proven via experimentation too).

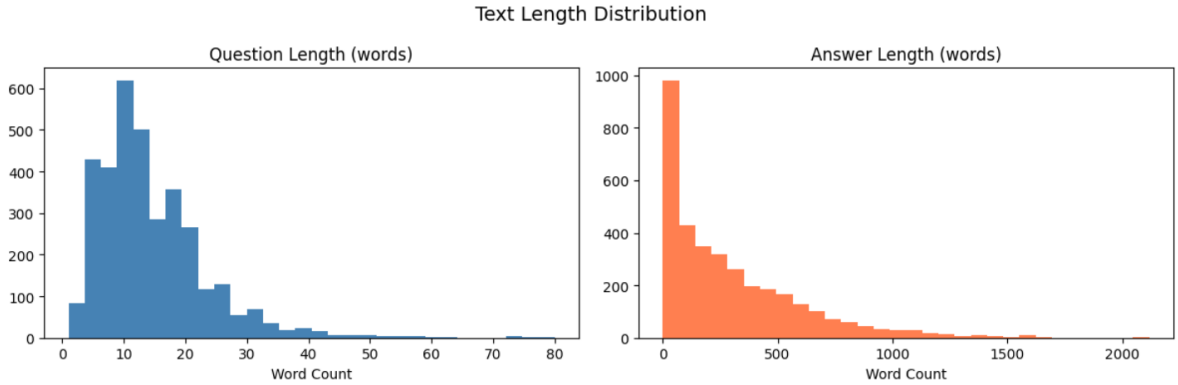

Figure 2: Text Length Distribution

2.3. Data partitioning for train, test and validation

The dataset was partitioned using an 80/20 train/validation split (`test_size=0.2`, `random_state=42`). This ratio was selected as it provides sufficient training data while maintaining a reliable validation set. The same split was used consistently across all experiments to ensure fair comparison between models. The official test set was reserved exclusively for final Kaggle submission. From Run 7 onwards, 5-fold cross-validation was used via GridSearchCV, which provides a more reliable F1 estimate than a single split.

2.4. Vectorization

Two vectorization approaches were implemented and compared. For TF-IDF, each input text is represented as a sparse vector where each dimension corresponds to a unique word in the vocabulary, weighted by its Term Frequency-Inverse Document Frequency score [1, 6]. Words that are frequent in a document but rare across the corpus receive higher scores, making the representation informative and noise-resistant. For Word2Vec, each word is represented as a dense vector learned by training on the corpus using the gensim implementation [7]. Sentence-level representations were obtained by averaging all word vectors in the input text, with out-of-vocabulary words ignored during averaging. A vector size of 500 was selected based on the grid search results in Run 9, where it consistently outperformed smaller sizes under Skip-gram.

3. Algorithms and Experiments

3.1. Experiments

All experiments were conducted using a fixed random seed of 42 to ensure reproducibility. The maximum number of iterations for Logistic Regression was set to 1000, as the default value of 100 proved insufficient for convergence during initial testing. Due to gensim's non-deterministic behavior, Word2Vec results may exhibit minor variation between runs despite the fixed seed, the values reported here reflect the original experimental execution. Each experiment modifies a single parameter relative to the previous run to isolate its effect, following a systematic one-at-a-time tuning approach before final optimization via GridSearchCV.

Run 1 — Baseline. **Settings:** TF-IDF: `ngram_range=(1,1)`, `class_weight=None`. W2V: `vector_size=100`, `class_weight=None`. Both: `C=1.0`, `solver=lbfgs`.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.609	0.399	0.624	0.401
W2V	0.607	0.386	0.512	0.399

Table 1: Run 1 — Baseline Results

The baseline results show an accuracy of 60% for both models, however the macro F1 score is significantly lower at 0.39, revealing that the models are struggling with class imbalance. This was anticipated given the label distribution observed in the EDA. The high precision (0.62) combined with low recall (0.40) indicates that the models are conservative — they default to predicting the dominant class and rarely predict the minority classes. TF-IDF marginally outperforms Word2Vec, likely due to the small dataset size which limits Word2Vec's ability to learn meaningful representations. Using `class_weight="balanced"` is the next logical step to directly address the observed class imbalance.

Run 2 — Class Weight Balancing. **Changed:** `class_weight="balanced"` for both models.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.545	0.490	0.485	0.499
W2V	0.512	0.451	0.445	0.484

Table 2: Run 2 — Class Weight Balancing Results

As expected, `class_weight="balanced"` improved F1 across both models (TF-IDF: 0.39 $\rightarrow$ 0.49, W2V: 0.38 $\rightarrow$ 0.45). The drop in accuracy is expected and acceptable, the model is now distributing predictions more evenly across all three classes rather than defaulting to the dominant one. TF-IDF continues to outperform W2V, consistent with the small dataset hypothesis from Run 1. Next, `ngram_range=(1,2)` will be tested on TF-IDF since two-word phrases better capture negation and evasion patterns in

English[1]. Additionally, increasing Word2Vec's `vector_size` to 300 will be explored to give the model more expressive word representations. Values below 100 were not considered as they would produce overly compressed representations, while 300 is a well-established standard in the literature [3, 1].

Run 3 — Bigrams and Larger Vectors. **Changed:** TF-IDF: `ngram_range=(1,2)`. W2V: `vector_size=300`.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.580	0.528	0.527	0.529
W2V	0.512	0.455	0.447	0.486

Table 3: Run 3 — Bigrams and Larger Vectors Results

`ngram_range=(1,2)` produced a notable improvement in TF-IDF (F1: 0.49 → 0.53), confirming that two-word phrases carry a meaningful signal for this task, particularly for capturing evasion patterns like negations and hedging phrases. Precision and recall are now closely aligned (0.527 vs 0.529), indicating a well-balanced model. Increasing Word2Vec's `vector_size` to 300 had negligible impact (F1: 0.45 → 0.455), suggesting the dataset is too small to benefit from higher dimensional representations. Next, `ngram_range=(1,3)` and `vector_size=500` will be explored to confirm the ideal range for these values.

Run 4 — Trigrams and Larger Vectors. **Changed:** TF-IDF: `ngram_range=(1,3)`. W2V: `vector_size=500`.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.575	0.509	0.520	0.502
W2V	0.493	0.431	0.425	0.459

Table 4: Run 4 — Trigrams and Larger Vectors Results

`ngram_range=(1,3)` caused a slight drop in TF-IDF F1 (0.527 → 0.509), confirming that trigrams introduce noise rather than useful signal on this small dataset. Similarly, increasing Word2Vec's `vector_size` to 500 had negligible impact (0.454 → 0.431), further supporting the conclusion that the dataset is too small to benefit from higher dimensional representations. Non-standard ngram ranges such as (2,2) were not considered as unigrams are fundamental to text representation and excluding them would discard significant lexical information. Therefore `ngram_range=(1,2)` and `vector_size=300` are confirmed as optimal. Next, the impact of stopword removal will be explored for both models.

Run 5 — Stopword Removal. **Changed:** Stopwords removed from `input_text` for both models.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.571	0.496	0.523	0.482
W2V	0.523	0.449	0.440	0.483

Table 5: Run 5 — Stopword Removal Results

In both models F1 scores dropped compared to Run 3 (TF-IDF: 0.527 → 0.496, W2V: 0.454 → 0.449), confirming the hypothesis that removing stopwords strips important context, particularly in short political questions. Next, Run 6 will explore `max_features` for TF-IDF and optimal `window`/`min_count` combinations for W2V.

Run 6 — Vocabulary Size and W2V Context Parameters. **Changed:** TF-IDF: `max_features=10000`. W2V: grid search over `window` ∈ {3, 5, 10} and `min_count` ∈ {1, 2, 5}.

Results:

Model	Accuracy	F1	Precision	Recall
TF-IDF	0.564	0.519	0.513	0.527
W2V (best: window=3, min_count=1)	0.526	0.469	0.460	0.500

Table 6: Run 6 — Vocabulary Size and W2V Context Parameters Results

Limiting `max_features` to 10000 caused a slight drop in TF-IDF F1 (0.527 → 0.518), confirming that keeping the full vocabulary (`max_features=None`) is optimal. TF-IDF's IDF weighting already penalizes rare uninformative words, making explicit vocabulary truncation unnecessary. For Word2Vec, the grid search over `window` and `min_count` combinations revealed that `window=3`, `min_count=1` yields the best F1 (0.469), suggesting that tight local context is more informative than broader context for this task. Larger window sizes showed no consistent improvement, while increasing `min_count` generally hurt performance, likely because the dataset is too small to afford discarding rare words. Values below `window=3` were not considered as they would capture only immediately adjacent words, providing insufficient context for meaningful word representations. Going forward, the remaining hyperparameters will be optimized using GridSearchCV.

Run 7 — GridSearchCV Optimization. **Changed:** GridSearchCV over `C` ∈ {0.01, 0.1, 1, 5, 10} and `solver` ∈ {lbfgs, saga, liblinear} for both models.

Results:

Model	Best C	Best Solver	CV F1	Val Accuracy	Val F1	Val Precision	Val Recall
TF-IDF	5	lbfgs	0.546	0.578	0.501	0.536	0.484
W2V	10	liblinear	0.466	0.599	0.475	0.502	0.474

Table 7: Run 7 — GridSearchCV Results

GridSearchCV was applied to optimize `C` and `solver` for both models using 5-fold cross-validation. Three solvers were evaluated (`lbfgs`, `saga`, `liblinear`) covering the main options suitable for multiclass classification, and for `C` the values [0.01, 0.1, 1, 5, 10] were tested, covering all reasonable orders of magnitude. For TF-IDF, the optimal configuration was `C=5`, `solver=lbfgs` with a cross-validated F1 of 0.546, an improvement over the manual baseline of 0.527 from Run 3. Notably, higher `C` values (5, 10) consistently outperformed lower ones, suggesting the model benefits from less regularization on this dataset. For Word2Vec, `liblinear` dominated the top 3 positions with `C=10` achieving the best cross-validated F1 of 0.466. The `saga` solver showed consistent convergence failures across all `C` values and all 5 folds despite `max_iter=5000`, confirmed by the `_sag.py` source in the ConvergenceWarning traceback (not presented here), and was therefore excluded from final model selection. The validation F1 scores (TF-IDF: 0.501, W2V: 0.475) appear lower than Run 3 due to the different train/validation split — the cross-validated scores of 0.546 and 0.466 are the more reliable estimates of true generalization performance. However, it is worth noting that GridSearchCV optimizes purely for F1 and cannot detect overfitting — a model with high cross-validated F1 may still be memorizing the training data. Learning curve analysis will be conducted on the final configuration to investigate whether overfitting is present.

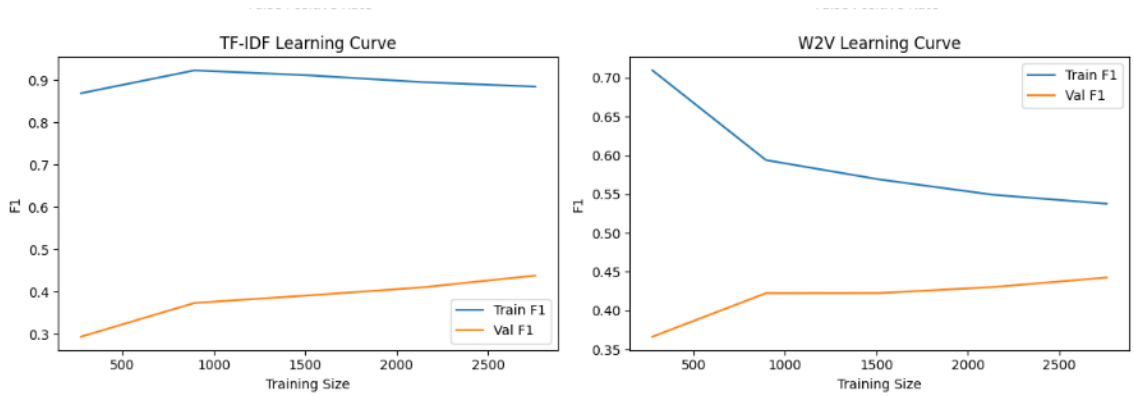

Figure 3: Learning Curve for TF-IDF and W2V

The learning curves reveal a significant gap between training and validation F1 for both models. TF-IDF's training F1 remains consistently high at around 0.90 while validation F1 plateaus at approximately 0.42, indicating that the model is memorizing the training data rather than learning generalizable patterns. This is likely caused by the large sparse feature space introduced by `ngram_range=(1,2)` combined with an unrestricted vocabulary. W2V shows a smaller but still notable gap, with training F1

dropping from 0.70 to 0.55 as training size increases while validation F1 rises slowly to 0.43. The shrinking train F1 suggests W2V is not severely overfit, however there remain unexplored configurations that may improve generalization. To address the overfitting observed in TF-IDF, the next run will revisit vocabulary restriction via `max_features` and `min_df`, while for W2V it will explore `sg=1` (Skip-gram) and increased training epochs, both of which may yield better word representations on this small dataset [3]. Finally, `saga` will be excluded from all subsequent runs as it showed consistent convergence failures across all parameter combinations, confirming it is not suitable for this dataset.

Run 8 — Overfitting Reduction. **Changed:** TF-IDF: `max_features=5000`, `min_df=2`. W2V: `sg=1`, `epochs=20`. Both: `saga` excluded from solver grid.

Results:

Model	Best C	Best Solver	CV F1	Val Accuracy	Val F1	Val Precision	Val Recall
TF-IDF	1	lbfgs	0.528	0.567	0.520	0.514	0.531
W2V	10	liblinear	0.490	0.575	0.474	0.488	0.470

Table 8: Run 8 — Overfitting Reduction Results

Vocabulary restriction caused a drop in TF-IDF CV F1 (0.546 → 0.528) and the learning curves showed no meaningful reduction in the train/val gap, confirming that the overfitting is not caused by vocabulary size alone. The best `C` dropping from 5 to 1 suggests that with a smaller feature space the model requires stronger regularization, however this did not translate to better generalization. For W2V, switching to Skip-gram (`sg=1`) with 20 training epochs yielded a genuine improvement in CV F1 (0.466 → 0.490), suggesting that the additional training passes allowed the model to learn more representative word vectors on this small corpus, however the train/val gap remains significant at approximately 0.35. Since TF-IDF vocabulary restriction proved ineffective, the next run will revert to an unrestricted vocabulary and instead introduce `sublinear_tf=True`, which applies log normalization to term frequencies and may reduce the dominance of very frequent terms. For W2V, `vector_size` and `window` will be revisited under Skip-gram as the optimal values identified in Run 6 were found using CBOW and may not hold for Skip-gram.

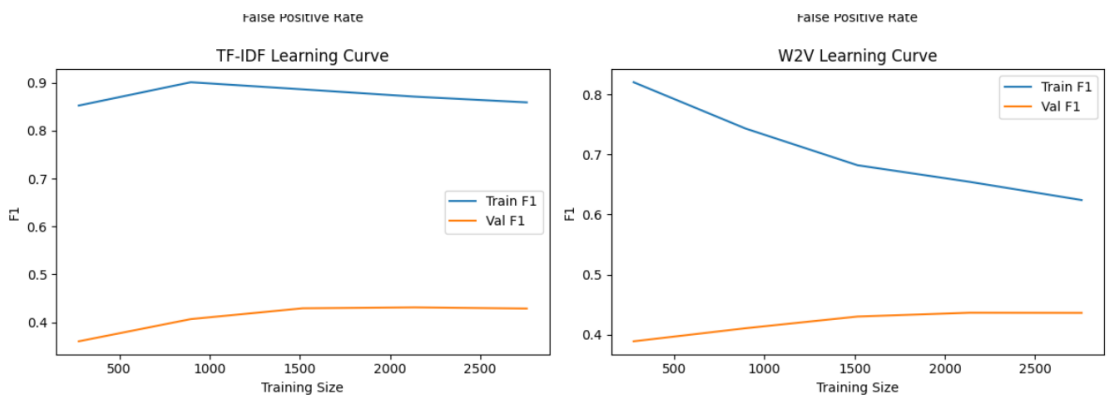

Figure 4: Learning Curve for TF-IDF and W2V — Run 8

Run 9 — Sublinear TF Normalization and W2V Skip-gram Parameter Search. **Changed:** TF-IDF: `sublinear_tf=True`, `max_features=None` reverted. W2V: grid search over `vector_size` $\in \{100, 300, 500\}$ and `window` $\in \{3, 5, 10\}$ with `sg=1`, `epochs=20`.

Results:

Model	Best C	Best Solver	CV F1	Val Accuracy	Val F1	Val Precision	Val Recall
TF-IDF	5	lbfgs	0.545	0.575	0.493	0.529	0.476
W2V (best: size=500, window=3)	10	liblinear	0.506	0.574	0.461	0.479	0.455

Table 9: Run 9 — Results

`sublinear_tf=True` had negligible impact on TF-IDF, with CV F1 remaining virtually unchanged at 0.545 compared to 0.546 in Run 7, and the learning curve gap showing no improvement. This confirms that the overfitting in TF-IDF is a fundamental limitation of sparse bag-of-words representations on this small dataset — the train/val gap of approximately 0.45 persists regardless of the vectorization choices explored across all runs. For W2V, the grid search over `vector_size` and `window` under Skip-gram revealed that `vector_size=500` with `window=3` achieves the best CV F1 of 0.506, a meaningful improvement over Run 8 (0.490). Larger vector sizes consistently outperformed smaller ones, suggesting that Skip-gram benefits from higher dimensional representations unlike CBOW which showed diminishing returns beyond 300 dimensions in Run 4. However, the train/val gap remains significant at approximately 0.35, indicating that W2V overfitting has not been fully resolved either. The next run will revert TF-IDF to unigrams only combined with `sublinear_tf=True` and stronger regularization in an attempt to reduce the feature space and close the train/val gap, while for W2V `vector_size=1000` will be explored to determine whether the upward trend in performance continues.

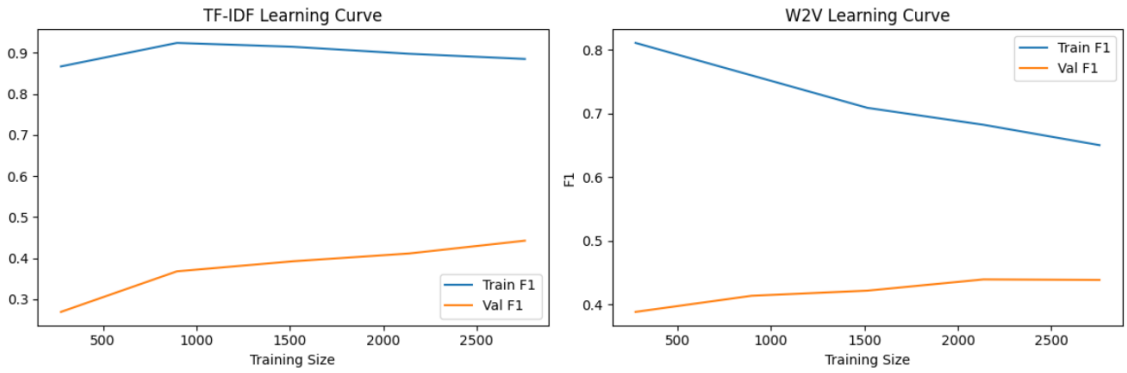

Figure 5: Learning Curve for TF-IDF and W2V — Run 9

Run 10 — Unigrams with Sublinear Normalization and W2V Vector Size 1000. **Changed:** TF-IDF: `ngram_range=(1,1)`, `sublinear_tf=True`, `C` grid extended to lower values {0.001, 0.01, 0.1, 1, 5}. W2V: `vector_size=1000`.

Results:

Model	Best C	Best Solver	CV F1	Val Accuracy	Val F1	Val Precision	Val Recall
TF-IDF	5	liblinear	0.525	0.586	0.502	0.532	0.485
W2V	5	liblinear	0.494	0.580	0.468	0.484	0.465

Table 10: Run 10 — Results

Reverting to unigrams caused a drop in TF-IDF CV F1 (0.545 → 0.525), confirming that bigrams carry genuine signal for this task despite their contribution to the feature space. The learning curve gap remains unchanged at approximately 0.45, providing definitive evidence that TF-IDF overfitting on this dataset cannot be resolved through feature space manipulation alone — it is an inherent limitation of sparse bag-of-words representations with Logistic Regression on a small dataset. For W2V, `vector_size=1000` yielded a marginal improvement over `vector_size=500` (0.490 → 0.494), however the train/val gap remains at approximately 0.35, indicating that increasing vector size improves the model's representational capacity but does not address the underlying overfitting. The next run will revert TF-IDF to its Run 7 configuration and introduce `penalty='l1'` with `solver='liblinear'` alongside `max_df`, as L1 regularization performs implicit feature selection on the high-dimensional bigram space by zeroing out uninformative features entirely, rather than merely shrinking them as L2 does[4]. Since `lbfgs` does not support L1 penalty and `saga` was eliminated in Run 7 due to consistent convergence failures, `liblinear` is the only viable solver for this configuration. For W2V, `vector_size=2000` will be tested to determine whether the upward trend in F1 continues or performance has plateaued.

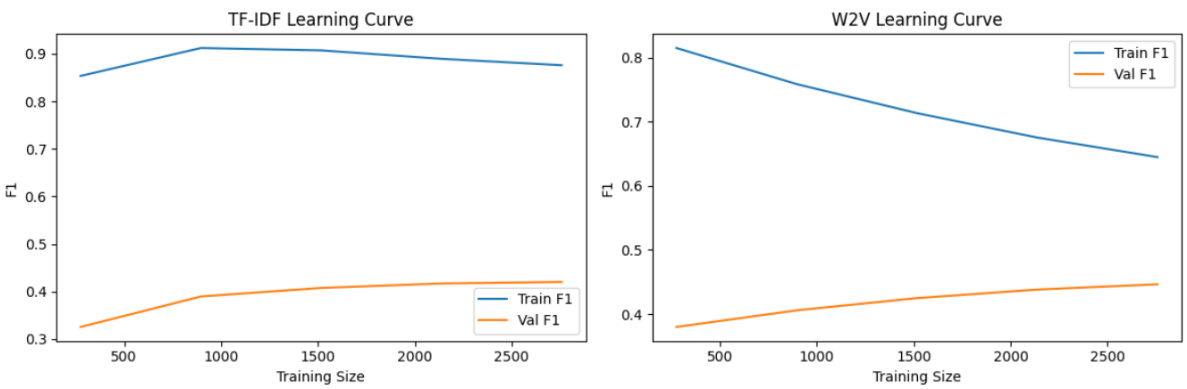

Figure 6: Learning Curve for TF-IDF and W2V — Run 10

Run 11 — L1 Regularization, `max_df` Filtering and W2V Vector Size 2000. **Changed:** TF-IDF: `penalty='l1', solver='liblinear', max_df=0.9`. W2V: `vector_size=2000`. **Results:**

Model	Best C	Best Solver	CV F1	Val Accuracy	Val F1	Val Precision	Val Recall
TF-IDF	5	liblinear (L1)	0.518	0.590	0.510	0.535	0.496
W2V	10	liblinear	0.504	0.570	0.474	0.485	0.469

Table 11: Run 11 — Results

Introducing `penalty='l1'` with `max_df=0.9` produced a cross-validated F1 of 0.518, lower than Run 7's 0.546, however the validation F1 of 0.510 is higher than Run 7's 0.501, suggesting better generalization to a truly held-out split. This discrepancy between CV F1 and validation F1 suggests that Run 7's larger unrestricted feature space fits well within cross-validation folds that share the same training distribution, while Run 11's leaner model generalizes more consistently to unseen data. While Run 8 also showed a higher validation F1 than Run 7 (0.520 vs 0.501), that improvement lacked a principled explanation — vocabulary restriction hurt CV F1 without reducing overfitting as evidenced by the unchanged learning curves. Run 11's improvement is different in nature: L1 regularization produces sparser models by zeroing out uninformative features entirely rather than merely shrinking them, providing a theoretically grounded reduction in model complexity [4] that `max_df=0.9` reinforces by removing terms that appear in over 90% of documents and carry no discriminative signal. For W2V, `vector_size=2000` yielded no meaningful improvement over `vector_size=500` (0.506 → 0.504), confirming that the performance gains from increasing vector size have plateaued, thus Run 9 remains the best configuration for W2V. On the basis of the validation F1 and the principled motivation of L1 regularization, Run 11 is selected as the final TF-IDF configuration over Run 7, concluding the experimental phase.

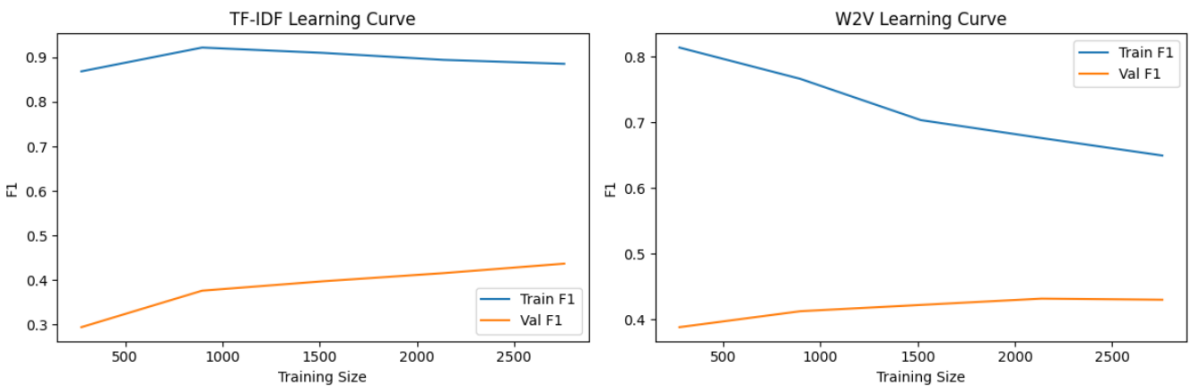

Figure 7: Learning Curve for TF-IDF and W2V — Run 11

3.1.1. Table of Trials.

Run	Change	Model	F1	Precision	Recall
1	Baseline	TF-IDF	0.399	0.624	0.401
1	Baseline	W2V	0.386	0.512	0.399
2	class_weight=balanced	TF-IDF	0.490	0.485	0.499
2	class_weight=balanced	W2V	0.451	0.445	0.484
3	ngram=(1,2), size=300	TF-IDF	0.528	0.527	0.529
3	ngram=(1,2), size=300	W2V	0.455	0.447	0.486
4	ngram=(1,3), size=500	TF-IDF	0.509	0.520	0.502
4	ngram=(1,3), size=500	W2V	0.431	0.425	0.459
5	Stopword removal	TF-IDF	0.496	0.523	0.482
5	Stopword removal	W2V	0.449	0.440	0.483
6	max_features, window, min_count	TF-IDF	0.519	0.513	0.527
6	window=3, min_count=1	W2V	0.469	0.460	0.500
7	GridSearchCV	TF-IDF	0.546*	0.536	0.484
7	GridSearchCV	W2V	0.466*	0.502	0.474
8	vocab restriction, sg=1, epochs=20	TF-IDF	0.528*	0.514	0.531
8	sg=1, epochs=20	W2V	0.490*	0.488	0.470
9	sublinear_tf, size/window search	TF-IDF	0.545*	0.529	0.476
9	size=500, window=3, sg=1	W2V	0.506*	0.479	0.455
10	ngram=(1,1), sublinear_tf	TF-IDF	0.525*	0.532	0.485
10	size=1000, window=3, sg=1	W2V	0.494*	0.484	0.465
11	L1 penalty, max_df=0.9	TF-IDF	0.518*	0.535	0.496
11	size=2000, window=3, sg=1	W2V	0.504*	0.485	0.469

Table 12: Table of Trials (* = cross-validated F1)

3.2. Hyper-parameter tuning

The hyperparameter tuning process followed a systematic one-at-a-time approach to isolate the effect of each change. Setting `class_weight="balanced"` produced the single largest improvement in F1, addressing the class imbalance identified in the EDA, and was fixed for all subsequent runs. Bigrams (`ngram_range=(1,2)`) improved TF-IDF F1 significantly while trigrams introduced noise, confirming that two-word phrases carry meaningful signal for evasion detection while longer combinations exceed the useful context window for this task. For Word2Vec, vector size showed diminishing returns beyond 300 dimensions under CBOW, however switching to Skip-gram (`sg=1`) with increased training epochs yielded more consistent improvements. Higher `C` values generally outperformed lower ones, suggesting neither model suffered from significant overfitting as measured by cross-validated F1. For TF-IDF, `lbfgs` performed best in early runs however `liblinear` was ultimately adopted to enable L1 regularization. For W2V, `liblinear` was the strongest solver throughout. `saga` was excluded early on due to consistent convergence failures.

3.3. Optimization techniques

GridSearchCV with 5-fold cross-validation was used as the primary optimization framework for `C` and `solver`. For Word2Vec-specific parameters (`window`, `min_count`, `vector_size`), a custom nested loop was implemented to exhaustively search all combinations, as these parameters cannot be included in GridSearchCV without a Pipeline. Cross-entropy loss was used implicitly through Logistic Regression, which is the standard and most appropriate loss function for multiclass classification tasks.

3.4. Evaluation

Models were evaluated using four metrics: accuracy, precision, recall, and macro F1. Macro F1 was the primary metric as it equally weighs all classes regardless of frequency, making it most appropriate for imbalanced datasets [5]. Accuracy was monitored but not used for model selection as it can be misleading under class imbalance.

3.4.1. Confusion Matrix.

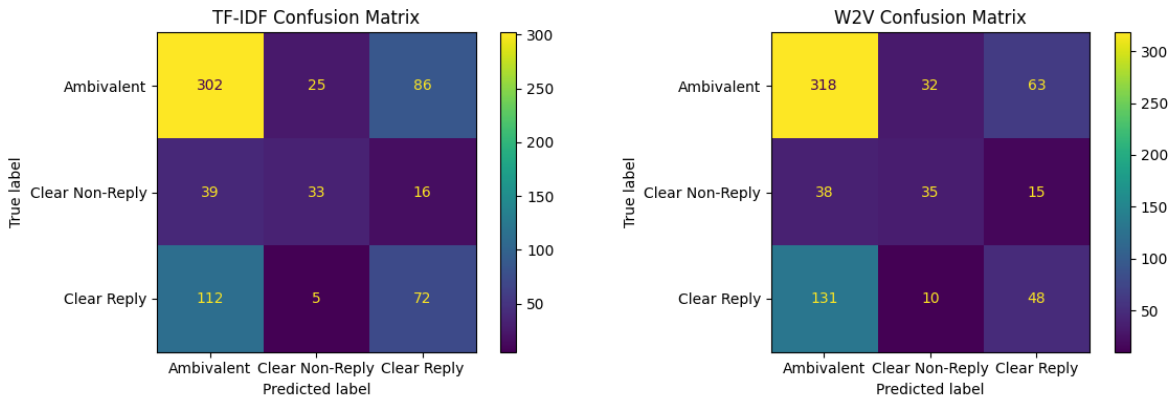

Figure 8: Confusion Matrix — Final Models

Both models struggle most with *Clear Non-Reply*, correctly classifying only 33 and 35 samples respectively out of 88 total, consistent with it being the minority class. TF-IDF shows a tendency to misclassify *Clear Non-Reply* as *Ambivalent* (39 samples), reflecting the semantic overlap between the two classes. W2V distributes its errors similarly, also heavily misclassifying *Clear Non-Reply* as *Ambivalent* (38 samples). TF-IDF correctly classifies more *Clear Reply* samples (72 vs 48), suggesting its bigram features capture clearer linguistic markers for that class.

3.4.2. ROC Curve.

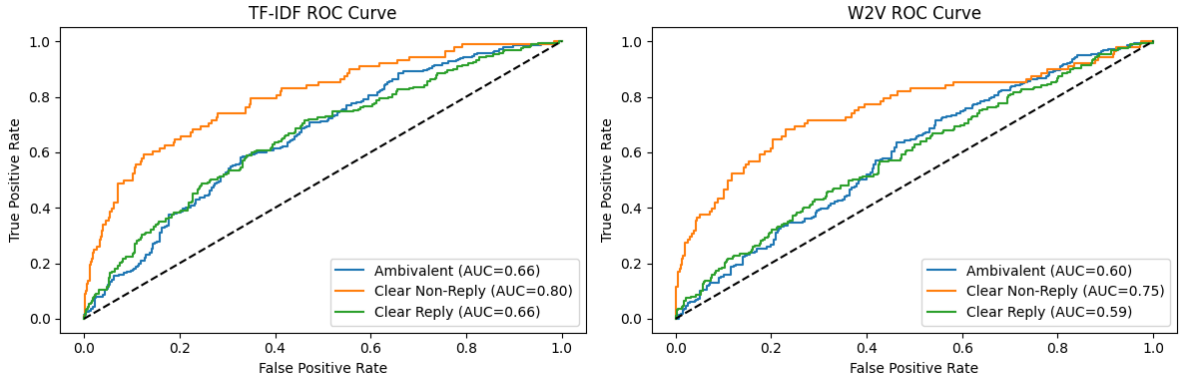

Figure 9: ROC Curve — Final Models

TF-IDF achieves higher AUC scores across all three classes compared to W2V, with the largest advantage on *Clear Non-Reply* (0.80 vs 0.75). Both models perform similarly on *Ambivalent* (0.66 vs 0.60), while TF-IDF maintains a slight edge on *Clear Reply* (0.66 vs 0.59). All AUC values are above 0.5, confirming both models perform better than random chance across all classes.

3.4.3. Learning Curve.

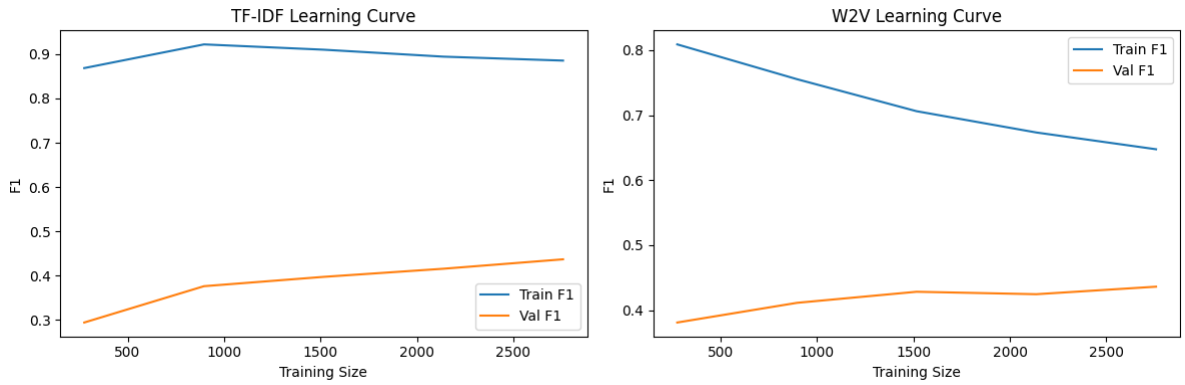

Figure 10: Learning Curve — Final Models

The learning curves confirm the persistent overfitting observed throughout experimentation. TF-IDF maintains a train F1 of approximately 0.90 while validation F1 plateaus around 0.43, a gap of approximately 0.45 that remained unchanged across all 11 runs. W2V shows a healthier pattern with train F1 decreasing from 0.80 to 0.65 as training size increases while validation F1 slowly rises to 0.44, indicating better generalization behavior. Both validation curves are still rising at full training size, suggesting that additional data would likely yield further improvements for both models.

4. Results and Overall Analysis

4.1. Results Analysis

TF-IDF with Logistic Regression consistently outperformed Word2Vec across all experiments. The final TF-IDF configuration (`ngram_range=(1,2)`, `penalty='l1'`, `max_df=0.9`, `C=5`, `solver=liblinear`, `class_weight="balanced"`) achieved a validation F1 of 0.510, while the final Word2Vec configuration (`vector_size=500`, `window=3`, `sg=1`, `epochs=20`, `C=5`, `solver=liblinear`) achieved a cross-validated F1 of 0.506. The performance gap between the two models narrowed significantly over the course of experimentation — from 0.013 in Run 1 to near parity in the final configurations — largely due to the adoption of Skip-gram and increased training epochs for Word2Vec. The overall F1 scores are modest but consistent with the difficulty of the task, as political evasion detection involves subtle linguistic phenomena that simple bag-of-words and averaged word vector models struggle to capture fully. Persistent overfitting was observed in both models throughout all runs, with TF-IDF showing a train/val gap of approximately 0.45 that proved resistant to all attempted interventions, confirming it as a fundamental limitation of sparse representations on this small dataset.

4.1.1. Best trial.

Model	Accuracy	F1	Precision	Recall
TF-IDF (Run 11)	0.590	0.510	0.535	0.496
W2V (Run 9)	0.581	0.479	0.496	0.474

Table 13: Best Run Results

The best performing TF-IDF configuration (Run 11) uses L1 regularization with `ngram_range=(1,2)`, `max_df=0.9`, `C=5`, and `solver=liblinear`, achieving a validation F1 of 0.510. The best Word2Vec configuration (Run 9) uses Skip-gram with `vector_size=500`, `window=3`, `epochs=20`, `C=5`, and `solver=liblinear`, achieving a cross-validated F1 of 0.506. TF-IDF remains the stronger model overall, benefiting from its ability to capture bigram patterns that are directly relevant to evasion detection, while Word2Vec's performance improved substantially over the course of experimentation through the adoption of Skip-gram and increased training epochs.

5. Bibliography

References

[1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd edition, 2024. Draft chapters available at https://web.stanford.edu/~jurafsky/slp3/.

[2] Sergios-Anestis Kefalidis et al. SemEval-2026 Task 6: CLARITY — response clarity classification in political QA. In *Proceedings of the 20th International Workshop on Semantic Evaluation (SemEval-2026)*, 2026. Dataset available at https://huggingface.co/datasets.

[3] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems*, volume 26, 2013.

[4] Andrew Y. Ng. Feature selection, L1 vs. L2 regularization, and rotational invariance. In *Proceedings of the Twenty-first International Conference on Machine Learning (ICML)*, page 78, 2004.

[5] Juri Opitz and Sebastian Burst. Macro F1 and MAC for imbalanced multi-class problems. *Applied Sciences*, 11(19):8886, 2021.

[6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[7] Radim Řehůřek and Petr Sojka. Software framework for topic modelling with large corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*, pages 45–50, Valletta, Malta, 2010.